



Amazon EC2 Elastic Load Balancing (ELB)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to Elastic Load Balancer (ELB)

What is a Load Balancer?

Physical hardware or virtual software that accepts incoming traffic and then distributes the traffic to multiple targets. They can **balance** the load via different rules. These rules vary based on the types of load balancers.



Elastic Load Balancer (ELB) is **a suite of load balancers** used to balance/distribute traffic to multiple EC2, ECS, Fargate and EKS instances

Application Load Balancer (ALB)

- operates on the OSI layer 7 the application layer (HTTP/HTTPS)
- Capable of routing based on HTTP information, can leverage Web Application Firewall (WAF)



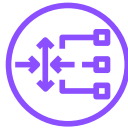
Network Load Balancer (NLB)

- operates on OSI layer 3/4 TCP/UDP
- Designed for large throughput of low-level traffic



Gateway Load Balancer (GWLB)

- Routes traffic to virtual appliances before traffic reaches its destination.
- Useful as a security layer for traffic in transit

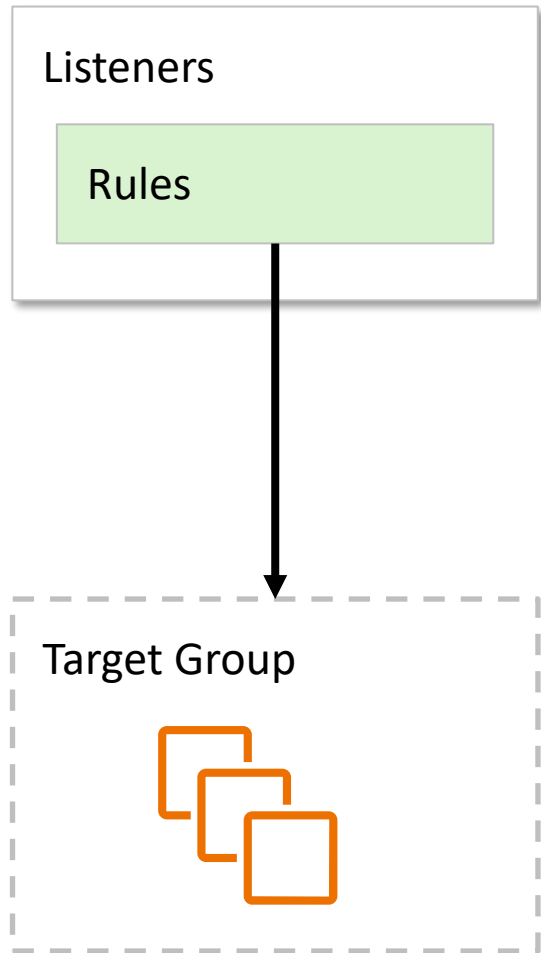


Classic Load Balancer (CLB)

- Operates on OSI layer 7 and 3/4
- Doesn't use target groups, directly attach targets
- Previous generation of load balancer, that is only used in legacy cases where clients have not moved over to ALB or NLB



ELB - The Rules of Traffic



Listeners

Incoming traffic is evaluated against listeners.

Listeners evaluate any traffic that matches the Listener's port.

Commonly listen on port 443 or port 80.

For Classic Load Balancer, EC2 instances are directly registered to the Load Balancer.

Rules (Only available for Application Load Balancer)

Listeners will then invoke rules to decide what to do with the traffic.

Generally, the next step is to forward traffic to a Target Group

More advance rules around HTTP request information is only for ALB since it operates on the OS. Layer.

Target Groups (Not available for Classic Load Balancer)

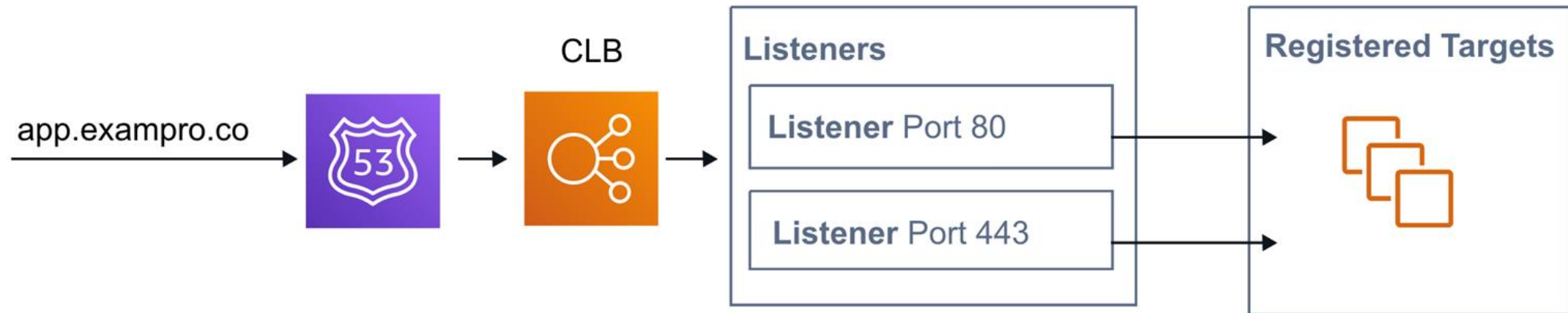
Are a logical grouping of possible targets such as specific EC2 instances, IP addresses

CLB just directly associates targets to the load balancer.

ELB - The Rules of Traffic

For Classic Load Balancer (CLB) traffic is sent to the Listeners.

When the port matches, it forwards the traffic to any EC2 instances that are registered to the Classic Load Balancer. CLB does not allow you to apply rules to listeners



Application Load Balancer (ALB)

- ALB is designed to balance HTTP and HTTPS traffic.
- It operate at Layer 7 (of the OSI Model).
- It's has a feature called Request Routing, which allows you to add routing rules to your listeners based on the HTTP protocol.
- Supports WebSockets and HTTP/2 for real-time, bidirectional communication applications.
- It can handle authorization and authentication of HTTP requests
- Can only be accessed via its hostname. If you need a static IP, forward an NLB to ALB
- AWS Web Application Firewall (WAF) can be placed in front of ALB for OWASP protection
- AWS Certificate Manager (ACM) can be attached to listeners to server custom domains over SSL/TLS for HTTPS
- Global Accelerator can be placed in front of ALB to improve global availability
- Amazon CloudFront can be placed in front of ALB to improve global caching of common HTTP requests.
- Amazon Cognito can be used to authenticate users via incoming HTTP requests

Use Cases:

- Microservices and Containerized Applications
- E-commerce and Retail Websites
- Corporate Websites and Web Applications
- SaaS Applications

OSI Layers

Layer 7 Application. eg. HTTP

Layer 6 Presentation eg. TLS

Layer 5 Session eg. NetBIOS

Layer 4 Transport eg. TCP/UDP

Layer 3 Network eg. IP, IPSec

Layer 2 Data Link eg. Ethernet

Layer 1 Physical eg. Ethernet

Network Load Balancer (NLB)

- NLB is designed to balance **TCP/UDP**.
- It **operate at Layer 4 (of the OSI Model)**
- It can handle **millions of requests per second** while still maintaining extremely low latency.
- Global Accelerator can be placed in front of ALB to improve global availability
- Preserves the client source IP
- When a static IP address is needed for a load balancer

Use cases:

- High-Performance Computing and Big Data Applications
- Real-Time and Multiplayer Gaming Platforms
- Financial Trading Platforms
- IoT and Smart Device Ecosystems
- Telecommunications Networks

OSI Layers

Layer 7 Application. eg. HTTP

Layer 6 Presentation eg. TLS

Layer 5 Session eg. NetBIOS

Layer 4 Transport eg. TCP/UDP

Layer 3 Network eg. IP, IPSec

Layer 2 Data Link eg. Ethernet

Layer 1 Physical eg. Ethernet

Classic Load Balancer (CLB)

- CLB is AWS's first load balancer (legacy)
- Can balance HTTP, HTTPS, or TCP traffic (not at the same time)
- It can use Layer 7-specific features (OSI Model) such as sticky sessions.
- It can also use strict Layer 4 (OSI Model) balancing for purely TCP applications.



Not recommended for use, instead use NLB or ALB

OSI Layers

Layer 7 Application. eg. HTTP

Layer 6 Presentation eg. TLS

Layer 5 Session eg. NetBIOS

Layer 4 Transport eg. TCP/UDP

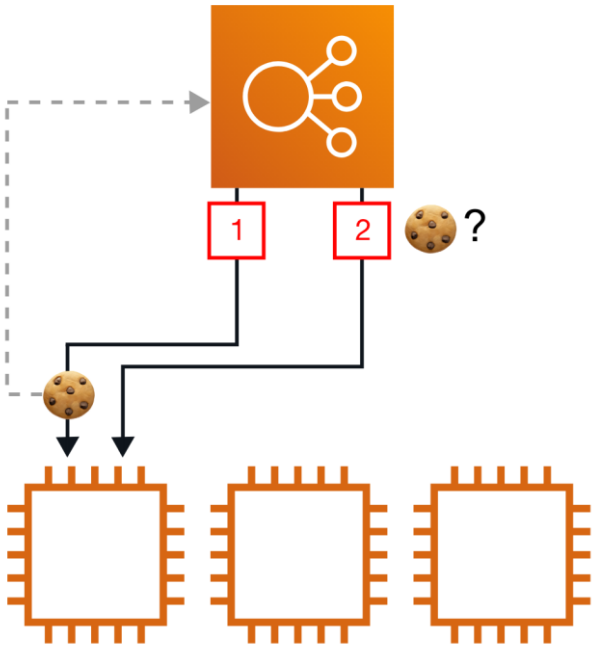
Layer 3 Network eg. IP, IPSec

Layer 2 Data Link eg. Ethernet

Layer 1 Physical eg. Ethernet

ELB - Sticky Sessions

Sticky Sessions (also known as session affinity) is an advanced load-balancing method that allows you to **bind a user's session to a specific compute targets**



- Ensures all requests from that session are sent to the same instance.
- Typically utilized with a Classic Load Balancer
- Can be enabled for ALB
 - can only be set on a Target Group, not individual EC2 instances.
- Cookies are used to remember which EC2 instance for returning traffic
 - ensures that all requests from the user during the session are sent to the same target.
- Useful when specific information is only stored locally on a single instance.
- To use sticky sessions, the client must support cookies.
- Sticky sessions are not supported if cross-zone load balancing is disabled.
- For application-based cookies names have to be specified per target group
 - Application-based stickiness does not work with weighted target groups.
- For duration-based cookies AWSALB is the only name used across all target groups
- WebSocket connections are inherently sticky.
- ALBs use the **Expires** attribute in the cookie header instead of the Max-Age attribute.
- ALBs do not support cookie values that are URL encoded.

ELB - Sticky Sessions

enable **duration-based stickiness** using the AWS CLI

```
aws elbv2 modify-target-group-attributes \  
--target-group-arn ARN \  
--attributes \  
Key=stickiness.enabled,Value=true \  
Key=stickiness.lb_cookie.duration_seconds,Value=time-in-seconds
```

enable **application-based stickiness** using the AWS CLI

- Do not use AWSALB, AWSALBAPP, or AWSALBTG for the cookie name; they're reserved for use by the load balancer.

```
aws elbv2 modify-target-group-attributes \  
--target-group-arn ARN \  
--attributes \  
Key=stickiness.enabled,Value=true \  
Key=stickiness.type,Value=app_cookie \  
Key=stickiness.app_cookie.cookie_name,Value=my-cookie-name \  
Key=stickiness.app_cookie.duration_seconds,Value=time-in-seconds
```

ELB - Cross-Zone Load Balancing

Cross-Zone Load Balancing is when traffic is evenly distributed to all targets which improves traffic flow, fault tolerance, cost efficiency.

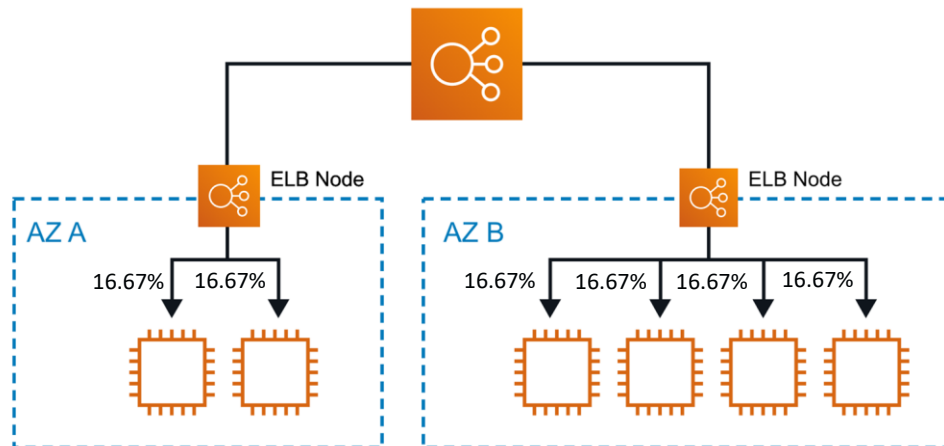
ALB — Turned on by default, cannot be turned off.

NLB — Turned on by default, can be turned on or off

CLB — Turned off by default, can be turned on or off

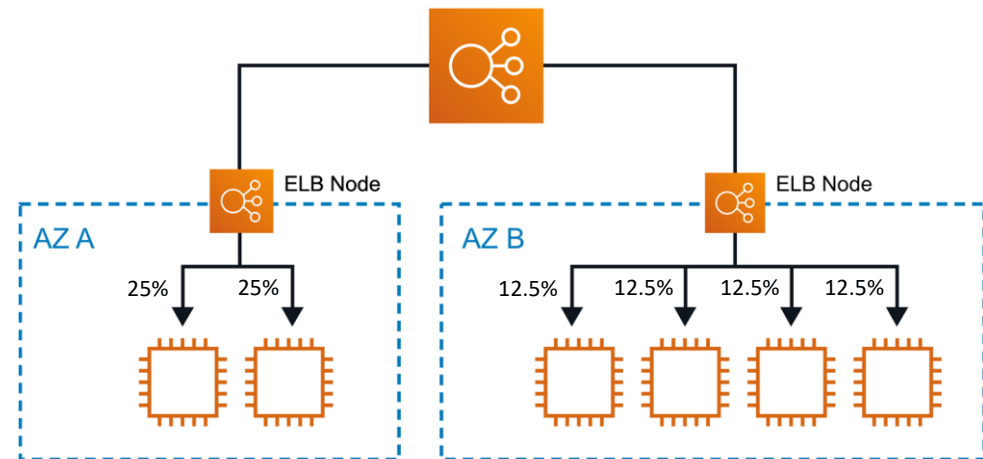
Cross-Zone Load Balancing **Enabled**

requests are distributed evenly across the instances **in all enabled** Availability Zones.



Cross-Zone Load Balancing **Disabled**

requests are distributed evenly across the instances **in only** its Availability Zone.



ELB - Cross-Zone Load Balancing

Turning on Cross-Zone balancing via the AWS CLI

```
aws elbv2 modify-target-group-attributes \  
--target-group-arn my-targetgroup-arn \  
--attributes \  
Key=load_balancing.cross_zone.enabled,Value=true
```

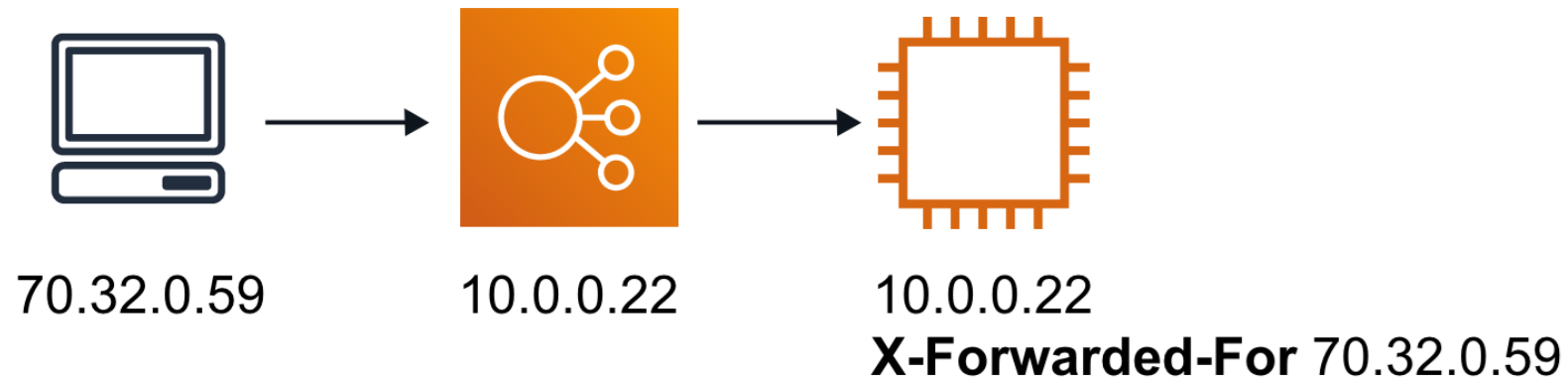
Turning off Cross-Zone balancing via the AWS CLI

```
aws elbv2 modify-target-group-attributes \  
--target-group-arn my-targetgroup-arn \  
--attributes \  
Key=load_balancing.cross_zone.enabled,Value=false
```

X-Forwarded-For (XFF) Header

If you **need the IPv4 address** of a user, check the **X-Forwarded-For** header

The **X-Forwarded-For (XFF)** header is a command method for identifying the **originating IP address** of a client connecting to a web server through an HTTP proxy or a load balancer.

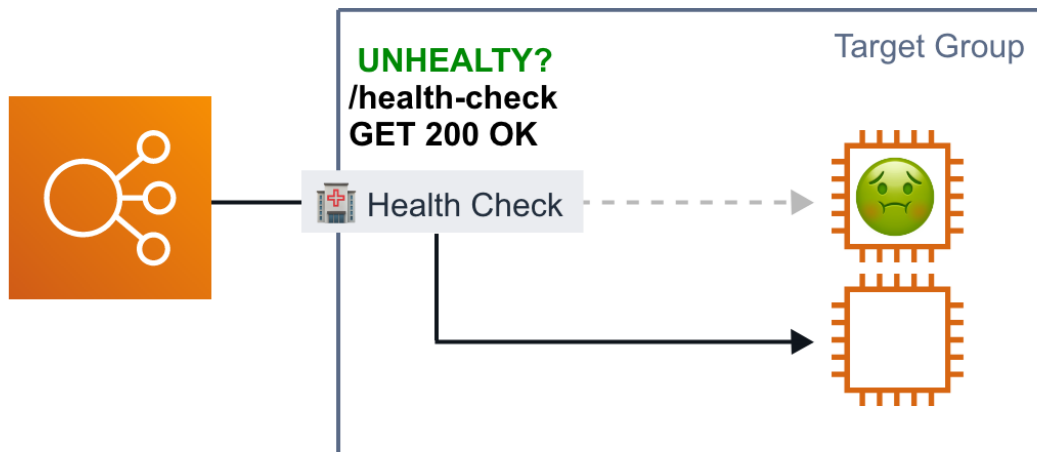


ELB - Health Checks

Instances that are monitored by the Elastic Load Balancer (ELB) **report back** Health Checks as **InService**, or **OutOfService**

Health Checks communicate directly with the instance to determine its state.

ELB **does not terminate (kill) unhealthy instance**. It will just redirect traffic to healthy instances



For ALB and NLB the Health checks are found on the **Target Group**

Targets Health checks Monitoring Tags

Protocol	i	HTTP
Path	i	/health-check
Port	i	traffic port
Healthy threshold	i	2
Unhealthy threshold	i	2
Timeout	i	5
Interval	i	10
Success codes	i	200

Edit health check

ALB – Rules

A rule is **composed of the following parts**:

Rule Conditions:

- Determine the conditions that the route should be considered.

Rules Action Types:

- Determines what action should be taken if the condition is met

Rule Priority:

- Determines the priority an rule should take over another.
 - Evaluated from **lowest to highest**

Rule Limits

100 total rules per Application Load Balancer

5 condition values per rule

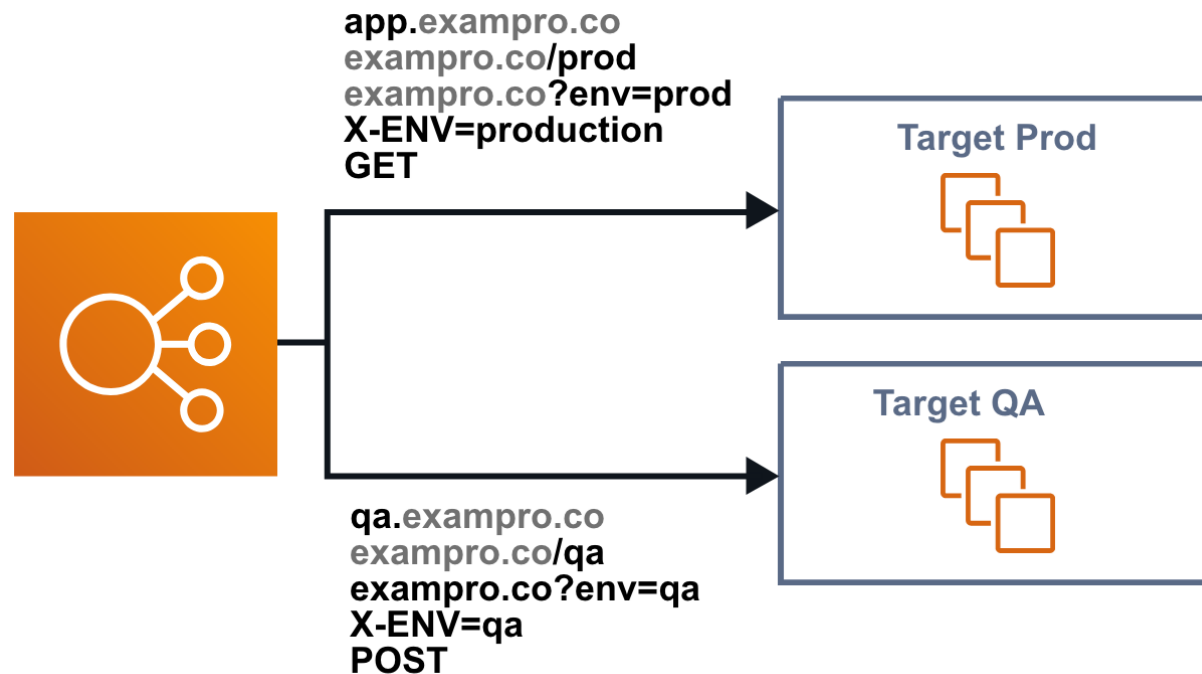
5 wildcards per rule

5 weighted target groups per rule

```
{
  "Conditions": [
    {
      "Field": "host-header",
      "HostHeaderConfig": {
        "Values": ["*.example.com"]
      }
    }
  ],
  "Actions": [
    {
      "Type": "forward",
      "TargetGroupArn": "..."
    }
  ],
  "Priority": 10,
  "ListenerArn": "..."
}
```

ALB - Request Routing

Request Routing rules allow you to forward or redirect traffic based on HTTP request information



For example we could have a two target groups for different environments and route based on sub domain, path, query string, http header, method and more.

ALB – Rule Action Types

ALB supports the following Rule Action Types

- **authenticate-cognito**
 - [HTTPS listeners] Use Cognito to authenticate requests.
- **authenticate-oidc**
 - [HTTPS listeners] Use an identity provider that is compliant with OIDC to authenticate requests.
- **fixed-response**
 - Return a custom HTTP response
- **forward**
 - Forward requests to the specified target groups.
- **redirect**
 - Redirect requests from one URL to another.



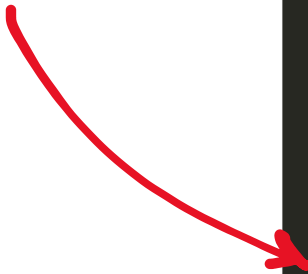
*If the protocol version is **gRPC** or HTTP/2, the only supported actions are forward actions.*

ALB Rules- Fixed Response

Fixed Responses drop requests and **return fixed data** for either 2XX, 4XX or 5XX Status Codes

When a fixed-response action is taken, the action and the URL of the redirect target are recorded in the access logs.

The count of successful fixed-response actions is reported in the HTTP_Fixed_Response_Count metric.



```
aws elbv2 create-rule \  
--listener-arn ... \  
--priority 5 \  
--conditions file://fixed-response.json \  
--actions Type=fixed-response,TargetGroupArn=...  
  
# fixed-response.json  
[  
  {  
    "Type": "fixed-response",  
    "FixedResponseConfig": {  
      "StatusCode": "200",  
      "ContentType": "text/plain",  
      "MessageBody": "Hello world"  
    }  
  }  
]
```

ALB Rules- Redirect actions

Redirect actions lets you **redirect client requests to another URL**.

You can specify either:

- Temporary redirects (HTTP 302)
- permanent redirects (HTTP 301)

You can retain information about the request:

- `{protocol}` - Use in the protocol and query components.
- `{host}` - Use in the hostname, path, and query components.
- `{port}` - Use in the port, path, and query components.
- `{path}` - Use in the path and query components.
- `{query}` - Use in the query component.

`{protocol}://{host}:{port}/{path}?{query}`



```
aws elbv2 create-rule \  
--listener-arn ... \  
--priority 5 \  
--conditions file://redirect.json \  
--actions Type=fixed-response,TargetGroupArn=...  
  
# redirect.json  
[  
  {  
    "Type": "redirect",  
    "RedirectConfig": {  
      "Protocol": "HTTPS",  
      "Port": "443",  
      "Host": "{host}",  
      "Path": "/{path}",  
      "Query": "{query}",  
      "StatusCode": "HTTP_301"  
    }  
  }  
]
```

ALB Rules- Forward actions

Forward allows you to **forward to a single or multiple targets**.

If you specific multiple targets you need to supply a weight.

Target weight can be between 0-999

Weighted target groups does not guarantee that sticky sessions are honored

```
aws elbv2 create-rule \  
--listener-arn ... \  
--priority 5 \  
--conditions file://forward-weighted.json \  
--actions Type=forward,TargetGroupArn=...  
  
# forward-weighted.json  
[\  
  {  
    "Type": "forward",  
    "ForwardConfig": {  
      "TargetGroups": [\  
        {  
          "TargetGroupArn": "...",  
          "Weight": 10  
        }, {  
          "TargetGroupArn": "...",  
          "Weight": 20  
        }  
      ]  
    }  
  ]  
]
```

ALB Rules- Rule Condition Types

The Forward Action has the following **condition types**:

- **host-header** Route based on the host name of each request.
- **http-header** Route based on the HTTP headers for each request.
- **http-request-method** Route based on the HTTP request method of each request.
- **path-pattern** Route based on path patterns in the request URLs.
- **query-string** Route based on key/value pairs or values in the query strings.
- **source-ip** Route based on the source IP address of each request.



The condition types are on available for ALB as they perform conditions based on HTTP requests

ALB Rules- Rule Condition Types

host-header

Route based on the host name of each request.

examples

- www.example.com
- *.example.com
- * (matches 0 or more characters) Wildcard
- ? (matches exactly 1 character)

You must include at least one "." character.

```
{
  "Conditions": [
    {
      "Field": "host-header",
      "HostHeaderConfig": {
        "Values": ["*.example.com"]
      }
    }
  ],
  "Actions": [
    {
      "Type": "forward",
      "TargetGroupArn": "..."
    }
  ],
  "Priority": 10,
  "ListenerArn": "..."
}
```

ALB Rules- Rule Condition Types

http-header

Route based on the HTTP headers for each request.

- Accept: text/html
- Accept-Encoding: gzip, deflate
- Accept-Language: en-US,en;q=0.5
- Cache-Control: no-cache
- Connection: keep-alive
- Cookie: name=value
- Content-Length: 348
- Content-Type: application/json
- Host: www.example.com
- Origin: http://www.example.com
- Referer: http://www.example.com/previous
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
- Authorization: Basic YWxhZGRpbjpvGVuc2VzYW1l

```
[
  {
    "Field": "http-header",
    "HTTPHeaderConfig": {
      "HttpHeaderName": "User-Agent",
      "Values": ["*Chrome*", "*Safari*"]
    }
  }
]
```

ALB Rules- Rule Condition Types

http-request-method

Route based on the HTTP request method of each request.

Request Methods:

- POST
 - GET
 - PUT
 - PATCH
 - DELETE
 - Custom HTTP Method
-
- * (matches 0 or more characters) Wildcard
 - ? (matches exactly 1 character)

```
[
  {
    "Field": "http-request-method",
    "HttpRequestMethodConfig": {
      "Values": [ "GET" ]
    }
  }
]
```

ALB Rules- Rule Condition Types

path-pattern

Route based on path patterns in the request URLs.

HTTP path examples

- /img/
- /img/*/pics

gRPC path examples

- /package
- /package.service
- /package.service/method

- * (matches 0 or more characters) Wildcard
- ? (matches exactly 1 character)

```
[  
  {  
    "Field": "path-pattern",  
    "PathPatternConfig": {  
      "Values": ["/img/*"]  
    }  
  }  
]
```

ALB Rules- Rule Condition Types

query-string

Route based on key/value pairs or values in the query strings.

- * (matches 0 or more characters) Wildcard
- ? (matches exactly 1 character)

A query-string is additional information that is provided

on the end of the base url:

https://www.example.com/ **?hello=world&goodbye-moon**



```
[
  {
    "Field": "query-string",
    "QueryStringConfig": {
      "Values": [{
        "Key": "version",
        "Value": "v1"
      }, {
        "Value": "*example*"
      }]
    }
  }
]
```

ALB Rules- Rule Condition Types

source-ip

Route based on the source IP address of each request.

- The IP address must be specified in CIDR format
- You can use both IPv4 and IPv6 addresses.
- Wildcard characters are not supported
- You cannot specify the 255.255.255.255/32 CIDR for the source IP rule condition.

```
[
  {
    "Field": "source-ip",
    "SourceIpConfig": {
      "Values": [
        "192.0.2.0/24",
        "198.51.100.10/32"
      ]
    }
  }
]
```

ELB – Target Registration

Instances

Sends traffic to specific EC2 instances based on their Instance ID.

IP Addresses

Often microservice-based architecture need to send to IP Addresses

When using Fargate, ALB can only target compute via IP addresses

Supports IPv4 and IPV6 and IPv4 to IPv6 NAT

IPv4 has be one of the following CIDR Blocks formats to work:

- 10.0.0.0/8 (RFC 1918)
- 100.64.0.0/10 (RFC 6598)
- 172.16.0.0/12 (RFC 1918)
- 192.168.0.0/16 (RFC 1918)

Lambda Function

The target group can only route to a specific Lambda function.

Lambda does not have be deployed within your VPC.

Only ALB's target group can target a Lambda function

Application Load Balancer

Send traffic to ALB from a NLB or ALB

Could be used to send an internet-facing ELB to an internal-facing

ELB – Target Deregistration

What is Connection Draining? (deregistration delay)

When you deregister a target, the load balancer **waits until in-flight requests have completed**.

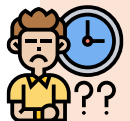
The status of a target is **draining** while connection draining is in progress.

Deregistration delay can be changed for ALB and CLB:

- Min: 1 second
- Max: 3600 seconds (1 hours)
- Default: 300 seconds (5 minutes)

NLB does not have configurable deregistration delay.

```
aws elbv2 modify-target-group-attributes \  
--target-group-arn ... \  
--attributes \  
Key=deregistration_delay.timeout_seconds,Value=600
```



When you deregister a target that was registered by IP address, you must wait for the deregistration delay to complete before you can register the same IP address again.

ELB – Trust Stores

ELB Trust Stores allow for Mutual Authentication (mTLS) using X509 Certificates for Application Load Balancer.